VOLUME 2

Intelligence Artificielle Avancée

De Praticien à Expert

> *"Les données sont le nouveau pétrole, mais le modèle est le moteur qui le raffine."*
>
> — **Pedro Domingos, *The Master Algorithm***

Réseaux **DL Avancé** **NLP/LLM** **Vision** **MLOps**

21 février 2026 | Suite du Volume 1

0 1 Table des matières

1 Récapitulatif du Volume 1 & Auto-évaluation **2**

2 Mathématiques Approfondies pour le Deep Learning **3**
2.1 Algèbre linéaire — ce qui se passe vraiment . . . 3
2.2 Calcul différentiel — la rétropropagation . . . 3
2.3 Fonctions d'activation . . . 5

3 Architectures de Réseaux de Neurones Avancées **6**
3.1 Réseaux Convolutifs (CNN) — Vision par ordinateur . . . 6
3.2 Transfer Learning — Ne réinventez pas la roue . . . 8
3.3 Transformers & Attention . . . 9

4 NLP Avancé & Grands Modèles de Langage (LLM) **12**
4.1 RAG — Retrieval-Augmented Generation . . . 12
4.2 Fine-tuning efficace avec LoRA . . . 13

5 Computer Vision Avancée **15**
5.1 Détection d'objets avec YOLO . . . 15
5.2 Segmentation sémantique . . . 16

6 MLOps — Du Notebook à la Production **18**
6.1 Suivi des expériences avec MLflow . . . 18
6.2 Déploiement avec FastAPI . . . 19
6.3 Containerisation avec Docker . . . 20

7 Projets Avancés & Défis Kaggle **22**
7.1 Roadmap des compétitions Kaggle . . . 22
7.2 Techniques avancées pour Kaggle . . . 22

8 Éthique et IA Responsable **25**
8.1 Détection et correction des biais . . . 25
8.2 Explicabilité avec SHAP . . . 26

9 Roadmap Avancée & Ressources Expert **27**
9.1 Plan 6 mois supplémentaires (mois 7 à 12) . . . 27
9.2 Papiers fondamentaux à lire . . . 27
9.3 Communautés et veille technologique . . . 28

10 Conclusion **29**

1 1 Récapitu du Volume 1 & Auto-évaluation

Avant d'aller plus loin, vérifiez que vous maîtrisez les bases du Volume 1.

Checklist Volume 1

Compétence	Outil	Niveau requis
Python (listes, dicts, OOP)	Python natif	★★★★○
Manipulation de données	Pandas / NumPy	★★★★○
Visualisation	Matplotlib / Seaborn	★★★○○
ML classique	Scikit-learn	★★★○○
Réseau de neurones simple	PyTorch ou Keras	★★○○○
Compte Kaggle actif	Kaggle	✔ Obligatoire

Attention

Si vous n'avez pas encore réalisé le projet **Titanic** sur Kaggle ou si les bibliothèques comme Pandas vous semblent encore floues, revenez au Volume 1 avant de continuer. Ce guide suppose que vous codez Python quotidiennement depuis au moins 4 semaines.

2 1 Mathéma
Approfondies pour le Deep Learning

2.1. Algèbre linéaire — ce qui se passe vraiment

Concept : Tenseurs et opérations

Un **tenseur** est la généralisation d'un tableau à N dimensions. C'est la structure de données fondamentale du deep learning :

- Scalaire = tenseur de dimension 0 (un nombre)
- Vecteur = tenseur de dimension 1 (une ligne)
- Matrice = tenseur de dimension 2 (tableau 2D)
- Tenseur 3D = ex : une image RGB (H × W × 3 canaux)
- Tenseur 4D = un batch d'images (N × H × W × C)

Formule

La transformation linéaire au cœur de chaque neurone :

$$\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

où $\mathbf{W} \in \mathbb{R}^{m \times n}$ est la matrice de poids, $\mathbf{x} \in \mathbb{R}^n$ le vecteur d'entrée, $\mathbf{b} \in \mathbb{R}^m$ le biais, et $\mathbf{y} \in \mathbb{R}^m$ la sortie.

2.2. Calcul différentiel — la rétropropagation

Formule

La règle de la chaîne est le moteur du deep learning. Pour $f(g(x))$:

$$\frac{df}{dx} = \frac{df}{dg} \cdot \frac{dg}{dx}$$

Dans un réseau à L couches, le gradient de la perte $\mathcal{L}$ par rapport aux poids de la couche l :

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(L)}} \cdot \prod_{k=l+1}^{L} \frac{\partial \mathbf{a}^{(k)}}{\partial \mathbf{a}^{(k-1)}} \cdot \frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{W}^{(l)}}$$

```python
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_deriv(x):
    s = sigmoid(x)
    return s * (1 - s)

# Reseau simple : 2 entrees -> 3 neurones -> 1 sortie
np.random.seed(42)
W1 = np.random.randn(3, 2) * 0.01    # Couche cachee
```

```python
b1 = np.zeros((3, 1))
W2 = np.random.randn(1, 3) * 0.01    # Couche de sortie
b2 = np.zeros((1, 1))

def forward(X):
    Z1 = W1 @ X + b1
    A1 = sigmoid(Z1)
    Z2 = W2 @ A1 + b2
    A2 = sigmoid(Z2)
    cache = (Z1, A1, Z2, A2)
    return A2, cache

def backward(X, Y, cache):
    m = X.shape[1]
    Z1, A1, Z2, A2 = cache

    # Gradient de la perte (Binary Cross-Entropy)
    dA2 = -(Y / A2) + (1 - Y) / (1 - A2)

    # Retropropagation couche 2
    dZ2 = dA2 * sigmoid_deriv(Z2)
    dW2 = (1/m) * dZ2 @ A1.T
    db2 = (1/m) * np.sum(dZ2, axis=1, keepdims=True)

    # Retropropagation couche 1
    dA1 = W2.T @ dZ2
    dZ1 = dA1 * sigmoid_deriv(Z1)
    dW1 = (1/m) * dZ1 @ X.T
    db1 = (1/m) * np.sum(dZ1, axis=1, keepdims=True)

    return dW1, db1, dW2, db2

# Donnees de test
X = np.random.randn(2, 100)
Y = (np.sum(X, axis=0) > 0).astype(float).reshape(1, -1)

# Une iteration
A2, cache = forward(X)
grads = backward(X, Y, cache)
print(f"Gradient W1 shape: {grads[0].shape}")  # (3, 2)
```

Listing 1 – Rétropropagation manuelle pour comprendre

2.3. Fonctions d'activation

Fonctions d'activation et leur usage

Activation	Formule	Quand l'utiliser
ReLU	$\max(0, x)$	Couches cachées, rapide, défaut
Leaky ReLU	$\max(\alpha x, x)$	Éviter les neurones morts
GELU	$x \cdot \Phi(x)$	Transformers (BERT, GPT)
Sigmoid	$\frac{1}{1+e^{-x}}$	Sortie binaire uniquement
Tanh	$\frac{e^x - e^{-x}}{e^x + e^{-x}}$	LSTM, RNN
Softmax	$\frac{e^{x_i}}{\sum_j e^{x_j}}$	Classification multi-classes

3 1 Architect de Réseaux de Neurones Avancées

3.1. Réseaux Convolutifs (CNN) — Vision par ordinateur

Concept : CNN et convolution

Un CNN apprend automatiquement des **filtres** (détecteurs de bords, textures, formes...) à partir des données. Les couches de convolution extraient des features locales, les couches de pooling réduisent la dimensionnalité.
Architecture typique : Input → [Conv → BN → ReLU → Pool] × N → Flatten → Dense → Output

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

# ====== Architecture CNN ======
class CNN(nn.Module):
    def __init__(self, num_classes=10):
        super(CNN, self).__init__()

        # Bloc 1 : 32 filtres
        self.bloc1 = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1),  # C_in, C_out, kernel
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.Conv2d(32, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(2),       # 32x32 -> 16x16
            nn.Dropout2d(0.25)
        )

        # Bloc 2 : 64 filtres
        self.bloc2 = nn.Sequential(
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.Conv2d(64, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2),       # 16x16 -> 8x8
            nn.Dropout2d(0.25)
        )

        # Bloc 3 : 128 filtres
        self.bloc3 = nn.Sequential(
            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.AdaptiveAvgPool2d(4)  # -> 4x4 quelle que soit l'
```

```
                entree
        )

        # Classifieur
        self.classifieur = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 4 * 4, 512),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(512, num_classes)
        )

    def forward(self, x):
        x = self.bloc1(x)
        x = self.bloc2(x)
        x = self.bloc3(x)
        return self.classifieur(x)

# ====== Entrainement ======
def entrainer(modele, train_loader, val_loader, epochs=30):
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    modele = modele.to(device)

    criterion = nn.CrossEntropyLoss(label_smoothing=0.1)
    optimizer = torch.optim.AdamW(modele.parameters(), lr=1e-3,
                                  weight_decay=1e-4)
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
        optimizer, T_max=epochs)

    historique = {'train_loss': [], 'val_acc': []}

    for epoch in range(epochs):
        # Entrainement
        modele.train()
        total_loss = 0
        for X, y in train_loader:
            X, y = X.to(device), y.to(device)
            optimizer.zero_grad()
            pred = modele(X)
            loss = criterion(pred, y)
            loss.backward()
            torch.nn.utils.clip_grad_norm_(modele.parameters(), 1.0)
            optimizer.step()
            total_loss += loss.item()

        # Validation
        modele.eval()
        correct = 0
        with torch.no_grad():
            for X, y in val_loader:
                X, y = X.to(device), y.to(device)
                pred = modele(X)
                correct += (pred.argmax(1) == y).sum().item()

        val_acc = correct / len(val_loader.dataset)
        historique['val_acc'].append(val_acc)
        scheduler.step()
```

```python

        if (epoch+1) % 5 == 0:
            print(f"Epoch {epoch+1:3d} | Loss: {total_loss/len(train_loader):.4f}"
                  f" | Val Acc: {val_acc:.2%}")

    return historique

# Usage
modele = CNN(num_classes=10)
print(f"Parametres : {sum(p.numel() for p in modele.parameters()):,}")
```

Listing 2 – CNN complet avec PyTorch pour la classification d'images

3.2. Transfer Learning — Ne réinventez pas la roue

Concept : Transfer Learning

Plutôt que d'entraîner un modèle de zéro (ce qui demande des millions d'images et des semaines de calcul), on **réutilise** un modèle pré-entraîné sur ImageNet et on l'adapte à notre tâche. C'est la technique la plus efficace en pratique.

```python
import torch
import torch.nn as nn
from torchvision import models, transforms
from torch.utils.data import DataLoader
from torchvision.datasets import ImageFolder

# ====== 1. Charger un modele pre-entraine ======
modele = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V2)

# ====== 2. Phase 1 : Feature extraction (geler les couches) ======
# On gele tout le reseau pour ne pas detruire les features apprises
for param in modele.parameters():
    param.requires_grad = False

# Remplacer uniquement la derniere couche
n_classes = 5   # Nombre de classes dans votre dataset
modele.fc = nn.Sequential(
    nn.Dropout(0.4),
    nn.Linear(modele.fc.in_features, 256),
    nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(256, n_classes)
)

# ====== 3. Transforms adaptes a ImageNet ======
train_transform = transforms.Compose([
    transforms.RandomResizedCrop(224, scale=(0.7, 1.0)),
    transforms.RandomHorizontalFlip(),
    transforms.ColorJitter(brightness=0.3, contrast=0.3, saturation=0.2),
    transforms.RandomRotation(15),
    transforms.ToTensor(),
```

```python
    transforms.Normalize(mean=[0.485, 0.456, 0.406],    # Stats ImageNet
                         std=[0.229, 0.224, 0.225])
])

val_transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])

# ====== 4. Phase 2 : Fine-tuning (degeler progressivement) ======
# Apres quelques epochs, on degel les dernieres couches
def debloquer_couches(modele, couches=['layer4', 'layer3']):
    for nom, param in modele.named_parameters():
        if any(c in nom for c in couches):
            param.requires_grad = True

    # LR plus faible pour les couches degelees (ne pas detruire les features)
    params = [
        {'params': [p for n, p in modele.named_parameters()
                    if 'fc' in n], 'lr': 1e-3},
        {'params': [p for n, p in modele.named_parameters()
                    if any(c in n for c in couches)], 'lr': 1e-5},
    ]
    return torch.optim.AdamW(params, weight_decay=1e-4)

# Phase 1 : entrainer seulement la tete
optimizer_phase1 = torch.optim.AdamW(
    filter(lambda p: p.requires_grad, modele.parameters()),
    lr=1e-3)

# ... entrainer 5-10 epochs ...

# Phase 2 : fine-tuning
optimizer_phase2 = debloquer_couches(modele)
# ... entrainer encore 10-20 epochs avec LR reduit ...
```

Listing 3 – Transfer Learning avec ResNet50 — technique pro

3.3. Transformers & Attention

Concept : Le mécanisme d'attention

Le mécanisme d'attention est la révolution qui a conduit à GPT, BERT, et tous les grands modèles de langage. L'idée clé : au lieu de traiter chaque token indépendamment, chaque token "regarde" tous les autres tokens et pondère leur importance.

X¹ Formule

Le Self-Attention (Scaled Dot-Product Attention) :

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

où $\mathbf{Q}$ (Queries), $\mathbf{K}$ (Keys), $\mathbf{V}$ (Values) sont des projections linéaires de l'entrée, et d_k est la dimension des clés (facteur d'échelle pour stabiliser les gradients).

```python
from transformers import (AutoTokenizer, AutoModelForSequenceClassification,
                          TrainingArguments, Trainer)
from datasets import Dataset
import numpy as np
import evaluate

# ====== 1. Donnees ======
textes_train = [
    "Ce film est absolument magnifique !",
    "Je n'ai pas aime du tout, vraiment decevant.",
    "Une oeuvre masterpiece, je recommande.",
    "Perte de temps totale, a eviter.",
    "Tres bon film, acteurs excellents.",
]
labels_train = [1, 0, 1, 0, 1]  # 1=positif, 0=negatif

dataset = Dataset.from_dict({'text': textes_train, 'label': labels_train})
dataset = dataset.train_test_split(test_size=0.2)

# ====== 2. Tokenisation ======
modele_name = "camembert-base"   # BERT en francais !
tokenizer = AutoTokenizer.from_pretrained(modele_name)

def tokeniser(examples):
    return tokenizer(examples['text'], truncation=True,
                     padding='max_length', max_length=128)

dataset = dataset.map(tokeniser, batched=True)

# ====== 3. Modele pre-entraine ======
modele = AutoModelForSequenceClassification.from_pretrained(
    modele_name, num_labels=2,
    id2label={0: "NEGATIF", 1: "POSITIF"},
    label2id={"NEGATIF": 0, "POSITIF": 1}
)

# ====== 4. Configuration de l'entrainement ======
args = TrainingArguments(
    output_dir="./resultats",
    num_train_epochs=5,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=16,
    learning_rate=2e-5,
    weight_decay=0.01,
    warmup_ratio=0.1,
```

```
    evaluation_strategy="epoch",
    save_strategy="best",
    load_best_model_at_end=True,
    metric_for_best_model="f1",
    fp16=True,  # Acceleration GPU
    logging_steps=10,
)

# ====== 5. Metriques ======
metric = evaluate.load("f1")

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)
    return metric.compute(predictions=predictions, references=labels,
                          average='weighted')

# ====== 6. Entrainer ======
trainer = Trainer(
    model=modele,
    args=args,
    train_dataset=dataset['train'],
    eval_dataset=dataset['test'],
    compute_metrics=compute_metrics,
)
trainer.train()

# ====== 7. Inference ======
from transformers import pipeline
classifieur = pipeline("text-classification", model=modele,
                       tokenizer=tokenizer)
print(classifieur("Ce produit est vraiment excellent !"))
```

Listing 4 – Fine-tuning d'un BERT pour classification de texte

4 1 NLP Avancé & Grands Modèles de Langage (LLM)

4.1. RAG — Retrieval-Augmented Generation

Concept : RAG

Le RAG permet de connecter un LLM à votre propre base de connaissances. Au lieu de répondre uniquement depuis sa mémoire, le modèle **recherche** d'abord les informations pertinentes dans vos documents, puis génère une réponse basée sur ces informations. C'est la technique qui alimente la plupart des chatbots d'entreprise.

```python
# pip install langchain faiss-cpu sentence-transformers openai
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.vectorstores import FAISS
from langchain.embeddings import HuggingFaceEmbeddings
from langchain.chains import RetrievalQA
from langchain.llms import HuggingFacePipeline
from transformers import pipeline

# ====== 1. Charger et decouper vos documents ======
documents = [
    """Le chat est un mammifere carnivore de la famille des felid s.
    Il est l'un des animaux de compagnie les plus populaires au monde.""",
    """Python est un langage de programmation interprete, multi-paradigme.
    Cree par Guido van Rossum, il est tres utilise en data science.""",
    """Le machine learning est un sous-domaine de l'intelligence artificielle
    qui permet aux systemes d'apprendre automatiquement a partir des donnees."""
]

splitter = RecursiveCharacterTextSplitter(
    chunk_size=200, chunk_overlap=30)
chunks = splitter.create_documents(documents)
print(f"{len(chunks)} chunks crees")

# ====== 2. Creer les embeddings et la base vectorielle ======
# Modele d'embeddings gratuit et local
embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2")

vectorstore = FAISS.from_documents(chunks, embeddings)
vectorstore.save_local("ma_base_vectorielle")   # Sauvegarder

# Charger plus tard :
# vectorstore = FAISS.load_local("ma_base_vectorielle", embeddings)

# ====== 3. Creer le retriever ======
retriever = vectorstore.as_retriever(
```

```
    search_type="mmr",        # Maximum Marginal Relevance (diversite)
    search_kwargs={"k": 3}    # Recuperer les 3 chunks les plus pertinents
)

# ====== 4. Tester la recherche ======
question = "Qu'est-ce que le machine learning ?"
docs_pertinents = retriever.get_relevant_documents(question)
for i, doc in enumerate(docs_pertinents):
    print(f"Doc {i+1}: {doc.page_content[:100]}...")

# ====== 5. Pipeline RAG complet ======
# (avec un LLM local - mistral, llama, etc.)
llm_pipeline = pipeline(
    "text-generation",
    model="mistralai/Mistral-7B-Instruct-v0.2",
    max_new_tokens=256,
    device_map="auto"        # Utilise GPU si disponible
)
llm = HuggingFacePipeline(pipeline=llm_pipeline)

qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=retriever,
    return_source_documents=True,
    chain_type_kwargs={
        "prompt": """Utilisez uniquement le contexte suivant pour repondre.
        Contexte: {context}
        Question: {question}
        Reponse:"""
    }
)
```

Listing 5 – Pipeline RAG avec LangChain et FAISS

4.2. Fine-tuning efficace avec LoRA

Concept : LoRA — Low-Rank Adaptation

Fine-tuner un LLM de 7 milliards de paramètres est hors de portée sur une seule GPU. **LoRA** résout ce problème : au lieu de modifier tous les poids, on ajoute des matrices de bas rang $\Delta W = BA$ (avec $r \ll d$) qui représentent uniquement les changements. Résultat : 100× moins de paramètres à entraîner.

```
# pip install peft transformers datasets bitsandbytes
from peft import get_peft_model, LoraConfig, TaskType, prepare_model_for_kbit_training
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
import torch

# ====== 1. Charger en 4-bit (quantisation) ======
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
```

```
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16,
    bnb_4bit_use_double_quant=True
)

modele_name = "mistralai/Mistral-7B-v0.1"
modele = AutoModelForCausalLM.from_pretrained(
    modele_name,
    quantization_config=bnb_config,
    device_map="auto"
)
tokenizer = AutoTokenizer.from_pretrained(modele_name)
tokenizer.pad_token = tokenizer.eos_token

# ====== 2. Preparer pour LoRA ======
modele = prepare_model_for_kbit_training(modele)

lora_config = LoraConfig(
    task_type=TaskType.CAUSAL_LM,
    r=16,                    # Rang des matrices LoRA (plus eleve = plus expressif)
    lora_alpha=32,           # Facteur d'echelle (lora_alpha / r = scaling)
    target_modules=["q_proj", "v_proj", "k_proj", "o_proj"],  # Couches attention
    lora_dropout=0.05,
    bias="none"
)

modele = get_peft_model(modele, lora_config)

# Afficher les parametres trainables
trainable = sum(p.numel() for p in modele.parameters() if p.requires_grad)
total = sum(p.numel() for p in modele.parameters())
print(f"Parametres entrainables: {trainable:,} / {total:,} ({trainable/total:.2%})")
# Resultat typique : 0.1% a 1% des parametres totaux !
```

Listing 6 – Fine-tuning LoRA avec PEFT

5 1 Compute Vision Avancée

5.1. Détection d'objets avec YOLO

Concept : YOLO — You Only Look Once

YOLO est le standard industriel pour la détection d'objets en temps réel. Contrairement aux approches en deux étapes (proposer des régions puis les classifier), YOLO prédit directement les boîtes et les classes en un seul passage réseau.

```python
# pip install ultralytics
from ultralytics import YOLO
import cv2

# ====== Inference avec YOLOv8 pre-entraine ======
modele = YOLO('yolov8n.pt')     # n=nano, s=small, m=medium, l=large, x=xlarge

# Detection sur une image
resultats = modele('photo.jpg', conf=0.5, iou=0.45)
resultats[0].show()    # Afficher
resultats[0].save('detection.jpg')

# Afficher les detections
for r in resultats:
    for box in r.boxes:
        classe = modele.names[int(box.cls)]
        confiance = float(box.conf)
        x1, y1, x2, y2 = map(int, box.xyxy[0])
        print(f"{classe}: {confiance:.2%} | Boite: ({x1},{y1})-({x2},{y2})")

# ====== Entrainement sur votre propre dataset ======
# Structure des donnees attendue :
# dataset/
#    images/train/*.jpg
#    images/val/*.jpg
#    labels/train/*.txt    (format YOLO: classe cx cy w h normalises)
#    labels/val/*.txt
#    data.yaml

# Contenu de data.yaml :
yaml_content = """
path: ./dataset
train: images/train
val: images/val

nc: 3               # Nombre de classes
names: ['chat', 'chien', 'voiture']
"""

modele_custom = YOLO('yolov8s.pt')    # Partir d'un modele pre-entraine

resultats = modele_custom.train(
```

```
    data='data.yaml',
    epochs=100,
    imgsz=640,
    batch=16,
    lr0=0.01,
    lrf=0.01,
    momentum=0.937,
    weight_decay=0.0005,
    augment=True,
    hsv_h=0.015,     # Augmentation couleur
    hsv_s=0.7,
    degrees=10.0,    # Rotation
    flipud=0.0,
    fliplr=0.5,
    mosaic=1.0,      # Mosaic augmentation
    project='runs',
    name='mon_detecteur',
)

# Evaluer
metriques = modele_custom.val()
print(f"mAP50    : {metriques.box.map50:.4f}")
print(f"mAP50-95: {metriques.box.map:.4f}")
```

Listing 7 – Détection et entraînement YOLO avec Ultralytics

5.2. Segmentation sémantique

```
# pip install segment-anything
import torch
import numpy as np
from segment_anything import sam_model_registry, SamPredictor,
    SamAutomaticMaskGenerator
import cv2
import matplotlib.pyplot as plt

# Charger SAM
sam = sam_model_registry["vit_h"](checkpoint="sam_vit_h_4b8939.pth")
sam.to('cuda' if torch.cuda.is_available() else 'cpu')

# Mode 1 : Segmentation automatique (tout segmenter)
mask_generator = SamAutomaticMaskGenerator(
    model=sam,
    points_per_side=32,
    pred_iou_thresh=0.86,
    stability_score_thresh=0.92,
)

image = cv2.imread('image.jpg')
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
masks = mask_generator.generate(image_rgb)
print(f"{len(masks)} objets detectes")

# Mode 2 : Segmentation sur un point donne
predictor = SamPredictor(sam)
predictor.set_image(image_rgb)

# Cliquer sur un point pour segmenter l'objet
```

```
point = np.array([[300, 200]])          # Coordonnees (x, y)
labels = np.array([1])                  # 1=inclure, 0=exclure

masks, scores, _ = predictor.predict(
    point_coords=point,
    point_labels=labels,
    multimask_output=True               # Retourner 3 masques (
        differentes granularites)
)

# Meilleur masque
meilleur_mask = masks[np.argmax(scores)]
plt.imshow(image_rgb)
plt.imshow(meilleur_mask, alpha=0.5, cmap='viridis')
plt.title(f"Segmentation (score: {scores.max():.3f})")
plt.axis('off')
plt.show()
```

Listing 8 – Segmentation avec SAM (Segment Anything Model)

6 1 MLOps — Du Notebook à la Production

Concept : MLOps

Le MLOps (Machine Learning Operations) est l'ensemble des pratiques pour déployer, surveiller et maintenir des modèles en production. C'est la compétence qui différencie un data scientist junior d'un senior.

6.1. Suivi des expériences avec MLflow

```python
import mlflow
import mlflow.sklearn
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score
import pandas as pd

# Demarrer le serveur : mlflow ui  (puis aller sur localhost:5000)
mlflow.set_experiment("classification-cancer")

X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Tester plusieurs configurations
configurations = [
    {"n_estimators": 50,  "max_depth": 5,    "min_samples_split": 2},
    {"n_estimators": 100, "max_depth": 10,   "min_samples_split": 5},
    {"n_estimators": 200, "max_depth": None, "min_samples_split": 2},
]

for config in configurations:
    with mlflow.start_run(run_name=f"RF_n{config['n_estimators']}"):
        # Logger les hyperparametres
        mlflow.log_params(config)

        # Entrainer
        modele = RandomForestClassifier(**config, random_state=42)
        modele.fit(X_train, y_train)

        # Evaluer
        y_pred = modele.predict(X_test)
        y_proba = modele.predict_proba(X_test)[:, 1]

        metriques = {
            "accuracy": accuracy_score(y_test, y_pred),
            "f1":       f1_score(y_test, y_pred),
            "auc_roc":  roc_auc_score(y_test, y_proba)
        }

```

```
        # Logger les metriques
        mlflow.log_metrics(metriques)

        # Logger le modele
        mlflow.sklearn.log_model(modele, "random_forest")

        print(f"n={config['n_estimators']:3d} | "
              f"Acc: {metriques['accuracy']:.4f} | "
              f"AUC: {metriques['auc_roc']:.4f}")
```

Listing 9 – Tracking d'expériences avec MLflow

6.2. Déploiement avec FastAPI

```
# pip install fastapi uvicorn pydantic joblib
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, validator
from typing import List
import joblib
import numpy as np
import uvicorn

# ====== Charger le modele ======
modele = joblib.load('modele_entraine.pkl')
scaler = joblib.load('scaler.pkl')

# ====== Definir l'API ======
app = FastAPI(
    title="API de Prediction",
    description="API pour predire la qualite du vin",
    version="1.0.0"
)

class WinInput(BaseModel):
    fixed_acidity: float
    volatile_acidity: float
    citric_acid: float
    residual_sugar: float
    chlorides: float
    free_sulfur_dioxide: float
    total_sulfur_dioxide: float
    density: float
    pH: float
    sulphates: float
    alcohol: float

    @validator('alcohol')
    def alcohol_range(cls, v):
        if not (8 <= v <= 15):
            raise ValueError('Alcool doit etre entre 8 et 15')
        return v

class PredictionOutput(BaseModel):
    prediction: int
    probabilite: float
    label: str
    confiance: str

```

```python
@app.get("/")
def accueil():
    return {"message": "API de prediction de qualite du vin v1.0"}

@app.post("/predire", response_model=PredictionOutput)
async def predire(vin: WinInput):
    try:
        # Preparer les features
        features = np.array([[
            vin.fixed_acidity, vin.volatile_acidity, vin.citric_acid,
            vin.residual_sugar, vin.chlorides, vin.free_sulfur_dioxide,
            vin.total_sulfur_dioxide, vin.density, vin.pH,
            vin.sulphates, vin.alcohol
        ]])

        # Normaliser
        features_scaled = scaler.transform(features)

        # Predire
        prediction = int(modele.predict(features_scaled)[0])
        proba = float(modele.predict_proba(features_scaled)[0].max())

        return PredictionOutput(
            prediction=prediction,
            probabilite=proba,
            label="Bon vin" if prediction == 1 else "Vin ordinaire",
            confiance="Haute" if proba > 0.8 else "Moyenne" if proba > 0.6 else "Faible"
        )
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@app.get("/sante")
def sante():
    return {"statut": "operationnel", "modele": type(modele).__name__}

# Lancer : uvicorn api:app --reload --port 8000
# Documentation auto : http://localhost:8000/docs
```

Listing 10 – API REST pour votre modèle avec FastAPI

6.3. Containerisation avec Docker

```dockerfile
# Dockerfile
FROM python:3.11-slim

WORKDIR /app

# Copier les dependances
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copier le code et les artefacts
COPY api.py .
```

```
COPY modele_entraine.pkl .
COPY scaler.pkl .

# Exposer le port
EXPOSE 8000

# Lancer l'API
CMD ["uvicorn", "api:app", "--host", "0.0.0.0", "--port", "8000"]
```

Listing 11 – Dockerfile pour déployer votre modèle

```
# Construire l'image
docker build -t mon-modele-ia:v1 .

# Lancer le conteneur
docker run -d -p 8000:8000 --name api-ia mon-modele-ia:v1

# Tester
curl -X POST "http://localhost:8000/predire" \
     -H "Content-Type: application/json" \
     -d '{"fixed_acidity": 7.4, "volatile_acidity": 0.7,
          "citric_acid": 0.0, "residual_sugar": 1.9,
          "chlorides": 0.076, "free_sulfur_dioxide": 11.0,
          "total_sulfur_dioxide": 34.0, "density": 0.9978,
          "pH": 3.51, "sulphates": 0.56, "alcohol": 9.4}'

# Voir les logs
docker logs api-ia
```

Listing 12 – Commandes Docker essentielles

7 1 Projets Avancés & Défis Kaggle

7.1. Roadmap des compétitions Kaggle

Progression recommandée sur Kaggle

	Compétition	Type	Objectif
1	Titanic	Classif. binaire	Top 30%
2	House Prices	Régression	Top 20%
3	Digit Recognizer	Vision (CNN)	Top 15%
4	Natural Language Processing	NLP	Top 25%
5	Store Sales Forecasting	Time Series	Top 20%
6	Compétition mensuelle active	Variable	Top 10%

7.2. Techniques avancées pour Kaggle

```python
import pandas as pd
import numpy as np
from sklearn.model_selection import KFold, StratifiedKFold
from sklearn.metrics import roc_auc_score
import lightgbm as lgb
import xgboost as xgb
from catboost import CatBoostClassifier

# ====== 1. Cross-Validation robuste ======
def cross_validate_oof(modele_fn, X, y, n_splits=5):
    """Out-Of-Fold predictions - la technique de validation standard en Kaggle"""
    kf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=42)
    oof_predictions = np.zeros(len(y))
    scores = []

    for fold, (train_idx, val_idx) in enumerate(kf.split(X, y)):
        X_train, X_val = X[train_idx], X[val_idx]
        y_train, y_val = y[train_idx], y[val_idx]

        modele = modele_fn()
        modele.fit(X_train, y_train)

        oof_predictions[val_idx] = modele.predict_proba(X_val)[:, 1]
        score = roc_auc_score(y_val, oof_predictions[val_idx])
        scores.append(score)
        print(f"  Fold {fold+1}: AUC = {score:.5f}")

    cv_score = roc_auc_score(y, oof_predictions)
    print(f"CV AUC: {cv_score:.5f} (+/- {np.std(scores):.5f})")
    return oof_predictions, cv_score
```

```python

# ====== 2. Stacking / Ensemble ======
class Stacker:
    """Combiner plusieurs modeles par stacking"""

    def __init__(self, modeles_base, meta_modele, n_splits=5):
        self.modeles_base = modeles_base
        self.meta_modele = meta_modele
        self.n_splits = n_splits

    def fit_predict(self, X_train, y_train, X_test):
        kf = KFold(n_splits=self.n_splits, shuffle=True, random_state=42)

        # Matrice de meta-features pour le train
        meta_train = np.zeros((len(X_train), len(self.modeles_base)))
        meta_test = np.zeros((len(X_test), len(self.modeles_base)))

        for i, (nom, modele) in enumerate(self.modeles_base.items):
            print(f"\nEntrainement: {nom}")
            test_preds = np.zeros((len(X_test), self.n_splits))

            for fold, (tr_idx, val_idx) in enumerate(kf.split(X_train)):
                modele.fit(X_train[tr_idx], y_train[tr_idx])
                meta_train[val_idx, i] = modele.predict_proba(X_train[val_idx])[:, 1]
                test_preds[:, fold] = modele.predict_proba(X_test)[:, 1]

            meta_test[:, i] = test_preds.mean(axis=1)

        # Meta-modele
        self.meta_modele.fit(meta_train, y_train)
        return self.meta_modele.predict_proba(meta_test)[:, 1]

# Usage
modeles_base = {
    'lgb': lgb.LGBMClassifier(n_estimators=500, learning_rate=0.05),
    'xgb': xgb.XGBClassifier(n_estimators=500, learning_rate=0.05),
    'cat': CatBoostClassifier(iterations=500, learning_rate=0.05, verbose=0),
}
meta = lgb.LGBMClassifier(n_estimators=100)

# ====== 3. Feature Engineering avance ======
def feature_engineering(df):
    df = df.copy()

    # Ratios
    df['ratio_ab'] = df['feature_a'] / (df['feature_b'] + 1e-8)

    # Interactions polynomiales selectionnees
    df['interaction_ab'] = df['feature_a'] * df['feature_b']

    # Statistiques par groupe
```

```
    for col in ['categorie_1', 'categorie_2']:
        stats = df.groupby(col)['valeur_cible'].agg(
            ['mean', 'std', 'count']).reset_index()
        stats.columns = [col] + [f'{col}_{s}' for s in ['mean', 'std
            ', 'count']]
        df = df.merge(stats, on=col, how='left')

    # Target encoding avec cross-validation (eviter le leakage !)
    from category_encoders import TargetEncoder
    encoder = TargetEncoder(smoothing=10)
    # Encoder uniquement sur les donnees d'entrainement

    return df
```

Listing 13 – Techniques pro pour monter dans les classements

8 1 Éthique et IA Responsable

Concept : Pourquoi l'éthique est indispensable

Un modèle IA peut discriminer, amplifier des biais, violer la vie privée, ou générer des informations fausses. En tant que praticien, vous êtes responsable de l'impact de vos systèmes. L'Union Européenne a adopté l'AI Act qui impose des obligations légales selon le niveau de risque des systèmes IA.

8.1. Détection et correction des biais

```python
# pip install fairlearn
from fairlearn.metrics import (MetricFrame,
    demographic_parity_difference,
                                equalized_odds_difference)
from fairlearn.postprocessing import ThresholdOptimizer
from sklearn.metrics import accuracy_score, precision_score,
    recall_score
import pandas as pd

# ====== Analyser les disparites ======
# sensitive_features : ex. genre, ethnicite, age
metric_frame = MetricFrame(
    metrics={
        'accuracy':  accuracy_score,
        'precision': precision_score,
        'recall':    recall_score,
    },
    y_true=y_test,
    y_pred=y_pred,
    sensitive_features=genre_test    # Variable sensible
)

print("Metriques par groupe :")
print(metric_frame.by_group)

print(f"\nEcart d'equite (accuracy): {metric_frame.difference()['
    accuracy']:.4f}")
print(f"Ratio d'equite (accuracy): {metric_frame.ratio()['accuracy
    ']:.4f}")

# ====== Mesurer la parite demographique ======
dp_diff = demographic_parity_difference(
    y_true=y_test, y_pred=y_pred,
    sensitive_features=genre_test)
print(f"Difference de parite demographique: {dp_diff:.4f}")
# Idealement proche de 0 (< 0.1 acceptable)

# ====== Corriger les biais post-hoc ======
correcteur = ThresholdOptimizer(
    estimator=modele,
    constraints="demographic_parity",
    predict_method='predict_proba',
    objective="accuracy_score"
```

```
)
correcteur.fit(X_train, y_train, sensitive_features=genre_train)
y_pred_equitable = correcteur.predict(X_test, sensitive_features=
    genre_test)
```

Listing 14 – Analyser les biais avec Fairlearn

8.2. Explicabilité avec SHAP

```
# pip install shap
import shap
import matplotlib.pyplot as plt

# Creer l'explainer
explainer = shap.TreeExplainer(modele)    # Pour tree-based models
# explainer = shap.Explainer(modele)     # Pour tout type de modele

# Calculer les valeurs SHAP
shap_values = explainer(X_test)

# ====== Importance globale ======
plt.figure(figsize=(10, 6))
shap.summary_plot(shap_values, X_test, feature_names=feature_names,
                  plot_type="bar", show=False)
plt.title("Importance globale des features (SHAP)")
plt.tight_layout()
plt.savefig("shap_global.png", dpi=150, bbox_inches='tight')

# ====== Explication locale (une prediction) ======
idx = 42    # Observation a expliquer
shap.waterfall_plot(shap_values[idx])
# Montre comment chaque feature pousse la prediction
# vers le haut (+) ou vers le bas (-) par rapport a la baseline

# ====== Dependence plot ======
# Comment l'age influence les predictions, colore par le genre
shap.dependence_plot("age", shap_values.values,
                     X_test, interaction_index="genre")
```

Listing 15 – Comprendre les décisions du modèle avec SHAP

9 1 Roadmap Avancée & Ressources Expert

9.1. Plan 6 mois supplémentaires (mois 7 à 12)

Mois 7-8 : Spécialisation profonde

- Lire le papier *Attention Is All You Need* (Vaswani et al., 2017)
- Implémenter un Transformer from scratch en PyTorch
- Viser le rang **Expert** sur Kaggle (1 médaille argent ou 2 bronze)
- Déployer un modèle en production avec Docker + FastAPI

Mois 9-10 : Research & State-of-the-art

- Lire 2-3 papiers par semaine sur arxiv.org
- Reproduire un papier récent (Papers With Code : paperswithcode.com)
- Publier un notebook Kaggle qui atteint le Top 5%
- Contribuer à un projet open-source IA (Hugging Face, PyTorch...)

Mois 11-12 : Portfolio et carrière

- Créer une application IA complète (frontend + backend + modèle)
- Passer une certification : *Google Cloud ML Engineer* ou *AWS ML Specialty*
- Rédiger des articles techniques sur LinkedIn ou Medium
- Postuler à des rôles ML Engineer, Data Scientist, ou AI Researcher

9.2. Papiers fondamentaux à lire

Les papiers qui ont changé l'IA

Papier	Année	Contribution
Attention Is All You Need	2017	Transformer
BERT	2018	Pré-entraînement NLP
GPT-3	2020	Few-shot learning
ResNet	2015	Residual connections
AlphaFold 2	2021	Structure protéines
Stable Diffusion	2022	Génération d'images
InstructGPT	2022	RLHF pour LLM
LoRA	2021	Fine-tuning efficace

9.3. Communautés et veille technologique

Où rester à jour

Ressource	Description
arXiv cs.LG	Prépublications quotidiennes en ML
Papers With Code	Papiers + code + benchmarks
r/MachineLearning	Discussions et actualités
HF Blog	Articles sur les derniers modèles
Yannic Kilcher (YT)	Revues de papiers en profondeur
Distill.pub	Explications visuelles de recherche
The AI Edge	Newsletter hebdomadaire

10 1 Conclusi

Vous avez parcouru un long chemin

En terminant ces deux volumes, vous avez acquis :

- ✓ Une solide base mathématique et algorithmique
- ✓ La maîtrise des frameworks PyTorch et Hugging Face
- ✓ Les compétences en Computer Vision et NLP avancé
- ✓ Les techniques pro pour exceller sur Kaggle
- ✓ Les bases du MLOps pour passer de la recherche à la production
- ✓ Une conscience éthique sur l'impact de vos modèles

Vos 3 actions pour cette semaine

1. **Implémentez** le pipeline RAG du cours sur un jeu de documents personnels
2. **Inscrivez-vous** à une compétition Kaggle active et soumettez un premier résultat
3. **Lisez** le résumé du papier *Attention Is All You Need* sur arXiv

Le vrai apprentissage commence
quand vous construisez quelque chose
que personne d'autre n'a encore construit.

Ce document fait suite au **Volume 1 — Apprendre l'IA de Zéro**.
Consultez-le si vous avez besoin de revoir les fondamentaux.